

```

1  /* =====
2      fotomarker.js – Foto-Marker (Fotobank Huma-Num/FNP) und Bild-Popup
3
4      Additive Ebene über der Karte: an jeder Fotokoordinate ein Punkt mit hellem
5      Kern, der bei Hover den Titel zeigt und beim Klick ein Popup öffnet. Hängt
6      an keinem Erzählzustand – Bbox, Kartenoffset und der optionale Hinweis
7      kommen als Parameter.
8
9      ACHTUNG letzterFotoOffsetX/Y lesen mapOffsetX/Y BEIM LADEN – diese Datei
10     muss nach geo-projektion.js stehen, sonst ReferenceError.
11     ===== */
12
13     // DOM-Referenzen des Popups, in setup() geholt.
14     let fotoPopup, fotoPopupTitel, fotoPopupPlz, fotoPopupBild, fotoPopupBeschreibung;
15
16     let fotoMarkerListe = [];
17     // Zustand des zuletzt gezeichneten Frames, alle drei in draw() gesetzt.
18     let letzteActiveBbox = null;
19     let letzterFotoOffsetX = mapOffsetX, letzterFotoOffsetY = mapOffsetY;
20     // Auch von uebersichtsrouten.js genutzt, damit alle Klickziele gleich gross sind.
21     const FOTO_MARKER_TREFFER_RADIUS = 12;
22
23     // Merkt sich Bbox und Offset des Frames für den Treffertest in mousePressed().
24     // Eigene Funktion, weil draw() sie unbedingt ruft, zeichneFotoMarker() nicht.
25
26     // ACHTUNG in der Graph-Ansicht bleibt der Marker frisch, obwohl dort keine
27     // Marker gezeichnet werden: ein Klick auf die Stelle, wo einer auf der Karte
28     // läge, öffnet sein Popup. mousePressed() prüft nur letzteActiveBbox, nicht
29     // den Ansichtsmodus. Ungelöst, siehe docs/cleanup-log.md.
30     function merkeKartenlage(bbox, offsetX, offsetY) {
31         letzteActiveBbox = bbox;
32         letzterFotoOffsetX = offsetX;
33         letzterFotoOffsetY = offsetY;
34     }
35
36     // hinweis (optional): { titel, text, alpha } – hängt ein beschriftetes Label
37     // mit Zuführungslinie an genau den Marker mit diesem Titel. Zeitpunkt und
38     // Deckkraft bestimmt der Aufrufer, siehe draw() in sketch.js.
39     function zeichneFotoMarker(activeBbox, offsetX = mapOffsetX, offsetY = mapOffsetY, alphaMultipl =
1, hinweis = null) {
40         if (alphaMultipl <= 0) return; // keine Karte, also auch keine Marker
41         // Aussen so gross wie der grösste F-Wert-Punkt, der Kern so gross wie der
42         // kleinste. Abgeleitet statt fest, damit der Marker mitwandert, wenn sich
43         // die Punktgrössen ändern. Wie jene skaliert er nicht mit dem Zoom.
44         let punktGroessen = Object.values(FWERT_PUNKT_DURCHMESSER);
45         let aussenRadius = Math.max(...punktGroessen) / 2;
46         let kernRadius = Math.min(...punktGroessen) / 2;
47         // Tooltip und Hinweis erst nach der Schleife, sonst überzeichnet sie ein
48         // später gezeichneter Marker.
49         let unterCursor = null;
50         let hinweisLabel = null;
51
52         push(); // schreibt fillStyle direkt, wie zeichneFwertPunkte in kreisgrafik.js
53         noStroke();
54         let scheibe = (x, y, r, rgb) => {
55             drawingContext.fillStyle = `rgba(${rgb.r}, ${rgb.g}, ${rgb.b}, ${alphaMultipl})`;
56             drawingContext.beginPath();
57             drawingContext.arc(x, y, r, 0, TWO_PI);
58             drawingContext.fill();
59         };
60         fotoMarkerListe.forEach(f => {
61             let pos = lonLatToScreen(f.lon, f.lat, activeBbox, offsetX, offsetY);
62             let hover = dist(mouseX, mouseY, pos.x, pos.y) < FOTO_MARKER_TREFFER_RADIUS;
63             // Farbe bleibt beim Hover gleich – ein Wechsel ins Orange brächte genau
64             // die Verwechslung zurück, die FOTO_MARKER_FARBE vermeidet.
65             let skala = hover ? 1.5 : 1;

```

```
66     scheibe(pos.x, pos.y, aussenRadius * skala, FOTO_MARKER_FARBE_RGB);
67     scheibe(pos.x, pos.y, kernRadius * skala, FOTO_MARKER_KERN_FARBE_RGB);
68
69     if (hover) unterCursor = { titel: f.titel, pos };
70     if (hinweis && f.titel === hinweis.titel) {
71         // Auf die Seite mit mehr Platz, sonst läuft der Hinweis aus dem Bild.
72         let links = pos.x > width / 2;
73         hinweisLabel = {
74             ankerX: pos.x, ankerY: pos.y,
75             x: pos.x + (links ? -40 : 40), y: pos.y,
76             text: hinweis.text, farbe: null,
77             hilfslinie: true, links, alpha: hinweis.alpha,
78         };
79     }
80 });
81 pop();
82
83 // Dieselbe Beschriftungsroutine wie die Ortsnamen und die
84 // Kreisgrafik-Erklärung: gestrichelte Zuführungslinie inklusive.
85 if (hinweisLabel) zeichneKreisLabels([hinweisLabel]);
86
87 if (unterCursor) {
88     push();
89     noStroke();
90     textFont(SCHRIFT_SANS); // wie .annotation-tag
91     textStyle(BOLD);
92     textSize(11);
93     let label = unterCursor.titel || 'Foto ansehen';
94     let tw = textWidth(label) + 16;
95     // Dieselbe Farbe wie der Marker selbst, damit Punkt und Infobox als ein
96     // Element lesen.
97     fill(FOTO_MARKER_FARBE_RGB.r, FOTO_MARKER_FARBE_RGB.g, FOTO_MARKER_FARBE_RGB.b, 200 *
alphaMultipller);
98     rect(unterCursor.pos.x + 10, unterCursor.pos.y - 12, tw, 20, 4);
99     fill(255, 255 * alphaMultipller);
100    textAlign(LEFT, CENTER);
101    drawingContext.fillText(label, unterCursor.pos.x + 18, unterCursor.pos.y - 2);
102    pop();
103 }
104 }
105
106 function oeffneFotoPopup(f) {
107     fotoPopupTitel.textContent = f.titel || '';
108     fotoPopupPlz.textContent = f.plz || '';
109     fotoPopupBild.src = f.fotoUrl;
110     fotoPopupBild.alt = f.titel || '';
111     fotoPopupBeschreibung.textContent = f.beschreibung || '';
112     fotoPopup.classList.add('offen');
113 }
114
115 function schliesseFotoPopup() {
116     fotoPopup.classList.remove('offen');
117 }
```